

Travel Itinerary Planning & Booking API

Django REST Framework, WT Authentication, Role-Based & Object-Level Permissions

1. Project Overview

The Travel Itinerary Planning & Booking API is a Django REST Framework (DRF) application that enables authenticated users to discover destinations, create and manage travel itineraries, add accommodations and activities, manage travel budgets, make bookings, write reviews, and collaborate with travel companions.

The API will use JSON Web Token (JWT) authentication to secure private resources. Role-based and object-level permissions will be used to ensure that users can access and modify only the resources they are authorised to manage.

2. Entity Relationship Diagram (ERD)

The ERD below represents the database structure using table-style entities. PK = Primary Key and FK = Foreign Key. Relationships are shown between the related tables.

Entity Definitions

Entity	Fields
USER	id (PK), username, email, password, travel_style, preferred_destination, budget_preference
CATEGORY	id (PK), name, description
DESTINATION	id (PK), name, country, city, description, average_rating, is_active, category_id (FK)
ITINERARY	id (PK), title, start_date, end_date, total_budget, user_id (FK), destination_id (FK)
ITINERARY_ITEM	id (PK), item_type, date, notes, itinerary_id (FK)
ITINERARY_COMPANION	id (PK), role, itinerary_id (FK), user_id (FK)
ACCOMMODATION	id (PK), name, price_per_night, destination_id (FK)
ACTIVITY	id (PK), name, price, destination_id (FK)
BOOKING	id (PK), booking_type, status, booking_date, user_id (FK), accommodation_id (FK), activity_id (FK)
BUDGET	id (PK), total_amount, spent_amount, itinerary_id (FK)

REVIEW	id (PK), rating, comment, user_id (FK), destination_id (FK)
--------	--

3. API Endpoints

Resource	Methods	Purpose
/api/auth/	POST	Register/login users
/api/destinations/	GET	Browse and search destinations
/api/categories/	GET, POST, PUT, DELETE	Manage categories
/api/itineraries/	GET, POST, PUT, DELETE	Manage itineraries
/api/bookings/	GET, POST, PUT, DELETE	Manage bookings
/api/reviews/	GET, POST, PUT, DELETE	Manage reviews
/api/budgets/	GET, POST, PUT, DELETE	Manage itinerary budgets

4. Authentication Flow

Authentication will follow a JWT-based flow. Users first register or log in through the authentication endpoints. The API then issues access and refresh tokens. Protected requests must include a valid access token, after which authentication and permission checks are performed before access to the requested resource is granted.

User

|

v

Register / Login

|

v

Django Authentication

|

v

JWT Access + Refresh Tokens

|

v

Authenticated API Request

|

v

JWT Validation

|

v

Permission Check

|

v

API Resource

5. Permission Matrix

Resource	Anonymous	Authenticated User	Owner	Admin
Destinations	Read	Read	—	Full
Categories	Read	Read	—	Full
Itineraries	—	Create / Read	Full	Full
Bookings	—	Create / Read	Modify / Delete own	Full
Reviews	Read	Create	Modify / Delete own	Full
Budgets	—	—	Full for own itinerary	Full

Object-level permissions will be used for user-owned resources such as itineraries, bookings, reviews, and budgets. Administrative users will have broader management permissions where appropriate.

6. URL Structure

The API will use a consistent `/api/` base path. Django REST Framework routers will be used for ViewSets where appropriate, while function-based views and class-based views will also be implemented where required to demonstrate the relevant DRF approaches.

```
/api/  
├── auth/  
├── destinations/  
├── categories/  
├── itineraries/  
├── bookings/  
├── reviews/  
└── budgets/
```

7. Testing Strategy

Test Area	Testing Focus
Models	Test relationships, required fields, validation, and database constraints.
Serializers	Test valid and invalid input, validation rules, and relationship serialization.
Views / ViewSets	Test CRUD operations, HTTP status codes, filtering, searching, and pagination.
Authentication	Test registration, login, JWT access tokens, and refresh tokens.
Permissions	Verify that users cannot access or modify another user's private resources.
URLs	Test that every defined endpoint resolves correctly.
Integration	Test complete workflows such as creating an itinerary, adding a budget, and making a booking.
Regression	Run the complete test suite after major changes to ensure existing functionality remains stable.

The project will use Django's testing framework and/or pytest. Tests will be maintained alongside the relevant application components to keep the test suite organised and maintainable.

8. Implementation Considerations

- Use Django REST Framework serializers to control input validation and API representations.
- Use ViewSets and routers for standard resource-based CRUD operations where appropriate.
- Use object-level permission checks to enforce ownership of private resources.
- Apply filtering, searching, and pagination to destination and other suitable list endpoints.
- Validate itinerary dates, budget values, ratings, and booking status transitions.
- Use database constraints and model validation to maintain data integrity.
- Keep authentication, permissions, serializers, views, URLs, and tests modular so the project can be extended without tightly coupling components.

DATABASE ENTITY RELATIONSHIP DIAGRAM

USER	CATEGORY	DESTINATION	ITINERARY
id : INT PK	id : INT PK	id : INT PK	id : INT PK
username : VARCHAR	name : VARCHAR	name : VARCHAR	title : VARCHAR
email : VARCHAR	description : TEXT	country : VARCHAR	start_date : DATE
password : VARCHAR		city : VARCHAR	end_date : DATE
travel_style : VARCHAR		description : TEXT	total_budget : DECIMAL
preferred_destination : VARCHAR		average_rating : DECIMAL	user_id : INT FK
budget_preference : DECIMAL		is_active : BOOLEAN	destination_id : INT FK
		category_id : INT FK	

ITINERARY_COMPANY	ITINERARY_ITEM	ACCOMMODATION	ACTIVITY
id : INT PK	id : INT PK	id : INT PK	id : INT PK
role : VARCHAR	item_type : VARCHAR	name : VARCHAR	name : VARCHAR
itinerary_id : INT FK	date : DATE	price_per_night : DECIMAL	price : DECIMAL
user_id : INT FK	notes : TEXT	destination_id : INT FK	destination_id : INT FK
	itinerary_id : INT FK		

BOOKING	BUDGET	REVIEW
id : INT PK	id : INT PK	id : INT PK
booking_type : VARCHAR	total_amount : DECIMAL	rating : INT
status : VARCHAR	spent_amount : DECIMAL	comment : TEXT
booking_date : DATE	itinerary_id : INT FK	user_id : INT FK
user_id : INT FK		destination_id : INT FK
accommodation_id : INT FK		
activity_id : INT FK		